

INFORME_PRUEBAS_UNITARIAS

Informe de pruebas unitarias — Backend S.I.G.I

Integrantes: Hawk Durant, Emilio Jaramillo, Rodrigo Candia

Asignatura: Arquitectura de Software — Duoc UC

Fecha: Junio 2026

Repositorio: [Proyecto-SIGI-Backend](#)

1. Introducción

En este informe documentamos la estrategia de pruebas unitarias que aplicamos al backend del **Sistema Inteligente de Gestión de Incendios (S.I.G.I)** de la Municipalidad Valle del Sol. El objetivo fue validar la lógica de negocio de cada microservicio sin levantar Docker ni MySQL, usando **JUnit 5** y **Mockito**, y medir la cobertura con **JaCoCo** integrado en Maven.

Definimos un umbral mínimo del **70 % de cobertura de líneas** por módulo de negocio. Los servicios de infraestructura (`eureka` y `api-gateway`) quedaron fuera del chequeo estricto porque casi no tienen reglas de dominio propias.

2. Stack de testing y por qué lo elegimos

Herramienta	Versión	Rol en el proyecto	Motivo de la elección
JUnit 5	5.x (vía Spring Boot 3.5)	Framework de pruebas	Es el estándar en Spring Boot; soporta anotaciones claras (<code>@Test</code> , <code>@DisplayName</code>) y se integra con Maven Surefire.
Mockito	5.x	Simulación de dependencias	Nos permitió probar servicios y controladores sin base de datos real ni llamadas Feign/HTTP.
Spring Boot Test	3.5.14	<code>@WebMvcTest</code> , contexto parcial	Para probar controladores REST con <code>MockMvc</code> sin arrancar todo el microservicio.
JaCoCo Maven Plugin	0.8.12	Medición de cobertura	Genera reportes HTML en <code>target/site/jacoco/</code> y falla el build en <code>mvn verify</code> si no se cumple el mínimo.

Herramienta	Versión	Rol en el proyecto	Motivo de la elección
Maven Surefire	3.5.5	Ejecución de tests en build	Corre automáticamente en <code>mvn test</code> y <code>mvn verify</code> en los 11 módulos del reactor.
spring-security-test	3.5.x	Pruebas de controladores	Necesario en módulos con Spring Security para <code>@WebMvcTest</code> con filtros desactivados.

No usamos **Testcontainers** en las pruebas unitarias del día a día porque ralentizan el ciclo de desarrollo; la dependencia está declarada en algunos `pom.xml` para pruebas de integración futuras.

3. Configuración JaCoCo (archivos clave)

La configuración central está en el `pom.xml` raíz del multi-módulo:

- Propiedad `jacoco.line.minimum = 0.70` (70 %).
- Fase `prepare-agent` : instrumenta la JVM durante los tests.
- Fase `report` : genera HTML por módulo tras `mvn test` .
- Fase `check` en `verify` : valida el umbral por bundle.
- Fase `report-aggregate` en el POM padre: reporte consolidado en `target/site/jacoco-aggregate/` .

Cada microservicio declara el plugin JaCoCo heredando del padre. `eureka` y `api-gateway` tienen `jacoco.skip.check=true` .

4. Métricas de cobertura (última ejecución `mvn verify`)

Total de pruebas ejecutadas: 241

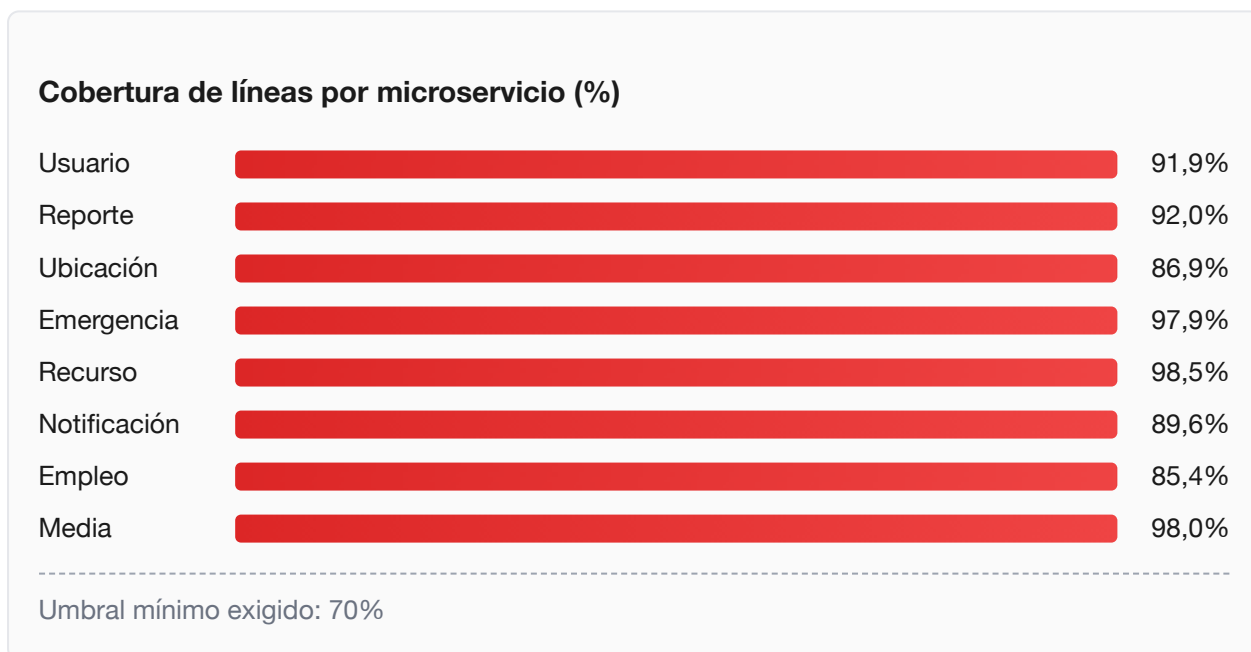
Cobertura de líneas (servicios de negocio): 92,4 % (1000 / 1082 líneas)

4.1 Tabla por microservicio

Módulo	Pruebas	Cobertura líneas	Cobertura instrucciones	Cobertura ramas	Cumple 70 %
servicio-usuario	57	91,9 % (215/234)	90 %	78 %	Sí
servicio-reporte	41	92,0 % (219/238)	89 %	71 %	Sí

Módulo	Pruebas	Cobertura líneas	Cobertura instrucciones	Cobertura ramas	Cumple 70 %
servicio-ubicacion	13	86,9 % (53/61)	80 %	70 %	Sí
servicio-emergencia	26	97,9 % (93/95)	95 %	100 %	Sí
servicio-recurso	24	98,5 % (130/132)	96 %	100 %	Sí
servicio-notificacion	13	89,6 % (60/67)	83 %	80 %	Sí
servicio-empleo	40	85,4 % (134/157)	85 %	90 %	Sí
servicio-media	27	98,0 % (96/98)	95 %	93 %	Sí

4.2 Gráfico de cobertura por líneas



La línea horizontal marca el **umbral exigido (70 %)**. Todos los servicios de negocio quedaron por encima.

5. Organización del código de pruebas

Las pruebas están en `src/test/java` de cada módulo, siguiendo el mismo paquete que el código productivo:

```

servicio-<nombre>/
src/test/java/com/sigi/servicio_<nombre>/
  service/          → lógica de negocio (Mockito puro)
  controller/       → endpoints REST (@WebMvcTest)
  dto/              → mapeos y getters/setters
  security/         → SecurityConfig, JwtUtil (usuario)
  config/           → inicializadores y migraciones
  model/            → entidades con lógica (@PrePersist, etc.)

```

5.1 Ejemplos representativos y resultados

Ejemplo A — Login y JWT (servicio-usuario)

- Clase: `UsuarioServiceTest.debeAutenticarUsuario`
- Qué valida: credenciales correctas, token generado por `JwtUtil`, respuesta `AuthResponseDTO`.
- Resultado: PASS — 19 pruebas en la clase de servicio.

Ejemplo B — Circuit breaker de ubicación (servicio-reporte)

- Clase: `ReporteServiceTest.sinCoordenadas`
- Qué valida: si OpenFeign/Resilience4j devuelve fallback, el reporte se guarda con latitud/longitud nulas.
- Resultado: PASS — el informe no se pierde aunque falle la geocodificación.

Ejemplo C — Prioridad por consenso (servicio-reporte)

- Clase: `PrioridadMetricasServiceTest`
- Qué valida: descripción `[INCENDIO]`, conteo de reportes similares en 24 h y elevación a CRÍTICA con umbral ≥ 10 .
- Resultado: PASS — 5 escenarios de prioridad.

Ejemplo D — Asignación de recursos (servicio-recurso)

- Clase: `RecursoServiceTest.asignaYpersiste`
- Qué valida: prioridad CRÍTICA pide 3 camiones y 2 brigadas; cada recurso pasa a estado ASIGNADO.
- Resultado: PASS — 11 pruebas de servicio.

Ejemplo E — Subida de archivos (servicio-media)

- Clase: `MediaServiceTest` con `@TempDir`
- Qué valida: guardado en disco, metadatos en JPA, rechazo de archivo vacío.
- Resultado: PASS — 13 pruebas de servicio.

6. Reportes HTML generados

Tras `mvn verify`, los reportes quedan en:

Módulo	Ruta del reporte
Cada servicio	<code>servicio-*/target/site/jacoco/index.html</code>
Consolidado	<code>target/site/jacoco-aggregate/index.html</code> (desde la raíz)

Copias de referencia para la entrega: carpeta `docs/cobertura/` (generar con el script de la guía).

7. Conclusiones del equipo

1. La suite de **241 pruebas** cubre servicios, controladores, DTOs, seguridad y configuración inicial sin depender de MySQL en CI local.
 2. **JaCoCo** nos da una métrica objetiva: el 92,4 % global de líneas supera ampliamente el 70 % pedido en la rúbrica.
 3. Los mocks de **OpenFeign** y **RestTemplate** nos permitieron simular la comunicación entre microservicios (reporte → emergencia → recurso/notificación) de forma aislada.
 4. Como mejora futura planteamos pruebas de integración con Testcontainers para validar queries JPA reales en MySQL 8.
-